

DWI Prompt Optimizer

Technical POC, Management Extension, and Governance Report

Evidence snapshot: 1 September 2026

Audience: engineering, architecture, product, delivery leadership, AI CoE, QA, and technical program stakeholders

Repository: vinodsalina-cpu/dwi

Functional branch: feature/po-05-product-hardening

POC implementation evidence baseline: 5088dbf6e5fdeea2415b991c429de8e5229d7a7b

Document status: final professional brief; local POC evidence plus explicitly proposed future direction

Scope boundary. Current implementation and future extension are separated throughout this report. PMS integration, sprint-informed token allocation, aggregated management feedback, production rollout, and real-provider semantic quality are not current product or qualification claims.

Executive summary

DWI is a post-login, consent-based project-initialization experience. Its first responsibility is to establish a persistent knowledge layer for the active project: identify the project root, disclose bounded inspection, collect only the evidence needed for a useful project brief, let the developer review and correct that brief, and persist the approved result. Prompt Optimizer builds on that reviewed project knowledge; it is not a generic prompt editor.

The current local POC standardizes a meaningful part of the AI interaction contract. It brings approved project knowledge, structured task input, deterministic source planning, assumptions, material-gap detection, reviewable composition, explicit human decisions, durable workspace-scoped state, and scoped reset into one controlled workflow. Prompt Optimizer is the sole visible product identity, Activity Bar container, native webview, and contributed open command. Home and Project Meta Context remain internal destinations within that surface.

The implementation baseline is 5088dbf6e5fdeea2415b991c429de8e5229d7a7b. At the documentation snapshot, local and live remote feature/po-05-product-hardening resolved to that same SHA with zero divergence; the exact-head GitHub **Verify DWI** run 33474904618 succeeded. The baseline records 421 passing repository tests, all six package typechecks, unchanged schema, a successful build and 387-file VSIX, same-process installed verification, narrow-layout review evidence, and preserved earlier installed restart/resume/reset qualification. The generated VSIX is 684,130 bytes with SHA-256 954ce0102754b79f7b3503bfe0c9053ba30ae21ab95177cc55d28c7643d2567b.

Three boundaries remain material:

- **Focused clarification is partial.** DWI detects and displays material questions, and domain code can represent and apply answers, but the installed product does not yet complete the focused one-question-at-a-time answer, skip, version-binding, and safe-application journey.
- **The latest cross-process continuity rerun is inconclusive.** The current layout correction could not obtain a fresh cross-process result because macOS System Events exposed no disposable VS Code window or sheet for the native consent click. No product assertion ran and no provider call occurred. Earlier qualified restart/resume/reset evidence remains preserved but is not silently reassigned to the later exact tree.
- **Real semantic improvement is unqualified.** The single authorized readiness request exceeded the frozen 12,000-token ceiling. Calibration and fixed-executor comparison did not start, and no retry is authorized.

Beyond the current POC, DWI offers a potential control point for consistent, governed AI-assisted delivery. After the baseline is stable, an optional PMS adapter could contribute minimal approved sprint and task attributes to a governance policy that produces a capped token-budget parameter for the internal composition engine. A separate consented, content-free feedback layer could aggregate outcome signals for developers, teams, SDLC operations, management, and an AI CoE. Budget would remain a bounded input rather than spending authority; feedback would support product and process improvement rather than employee surveillance.

Recommended conclusion

Use the current POC for a representative management demonstration and a structured usefulness assessment. Complete exact-tree continuity evidence and the focused clarification slice before treating the interaction baseline as stable. In parallel, define—but do not yet operationalize—the PMS adapter, budget-policy precedence, privacy-safe feedback event model, and pilot success measures. A bounded pilot should follow only after these controls are reviewed and offline or shadow allocation succeeds.

1. Scope, method, and evidence labels

1.1 Sources used

This report cross-checks management direction against the current repository and program records:

- AGENTS.md for the consent-first product direction;
- docs/prompt-optimizer-program/context.md for compact current truth;
- docs/prompt-optimizer-program/progress.md for completed, pending, and blocked items;
- docs/prompt-optimizer-program/expected.md for the active acceptance contract;
- docs/prompt-optimizer-program/operation-policy.md for authority and privacy invariants;
- docs/prompt-optimizer-program/phases/phase-5-outcome.md and phase-6-outcome.md for local POC outcomes;
- artifacts/vsix/developer-work-intelligence-0.1.0.evidence.json and the VSIX archive for exact package facts;
- current source for token estimates, feedback primitives, question handling, host persistence, and UI behavior;
- local Git metadata, live remote branch identity, and exact-head CI status.

This documentation task does not rerun product tests, call a provider, change a budget threshold, or perform merge, tag, release, or promotion. Document generation and document-format validation are separate from product qualification.

1.2 Evidence labels

Label	Meaning
Implemented	Source and current product records contain the capability. This label alone does not establish installed or production behavior.
Qualified locally	A named local test, package, installed, or evidence lane passed on a referenced candidate. It is not a production or release claim.
Partial	Some domain or UI support exists, but an intended end-user path or required proof is incomplete.
Preserved prior evidence	A prior candidate passed the named lane; a later exact tree did not obtain a fresh result.
Proposed future extension	Architecture or product direction requiring design, authorization, implementation, privacy review, and qualification.
Blocked	A frozen gate failed or stopped, and the governing contract does not authorize retry or waiver.

1.3 Implementation evidence snapshot

Item	Observation at the documentation snapshot
Repository	https://github.com/vinodsalina-cpu/dwi
Functional branch	feature/po-05-product-hardening
POC implementation baseline	5088dbf6e5fdeea2415b991c429de8e5229d7a7b
Functional upstream	origin/feature/po-05-product-hardening at the same baseline before this documentation commit
Upstream divergence	0 behind / 0 ahead before final documentation packaging
Relative to live origin/main	0 behind / 28 commits ahead
Exact-head CI	GitHub Verify DWI run 33474904618 passed
Tracked source	Clean at package evidence generation; unrelated local evidence files remain untracked
Phase state	Phase 5 local POC complete; Phase 6 local POC qualification complete; Phase 4 semantic qualification blocked

Exact-tree rule. Test, package, installed, and continuity evidence belongs to the exact candidate and artifact that produced it. A green result on an earlier package does not automatically qualify a later UI or navigation tree.

2. Product direction and operating contract

2.1 Consent-first initialized project knowledge

DWI’s first product journey is project initialization:

1. identify the project root;
2. disclose the bounded inspection scope;
3. obtain consent;
4. collect only the evidence needed for a useful project brief;
5. let the developer review, correct, and approve that brief;
6. persist the approved initialization state;
7. allow later task workflows to use the reviewed knowledge.

The initialized knowledge layer is the prerequisite for Prompt Optimizer. This boundary prevents the product from becoming a disconnected set of tools or a prompt editor that silently accumulates unreviewed context.

2.2 Current interaction contract

Approved Project Meta Context → structured task → deterministic source plan → assumptions and material gaps → local composition → developer review → save or scoped reset

Navigation through Input, Resolve, and Review is provider-free. A provider boundary is reached only through an explicit disclosed semantic action. If that action fails validation or is unavailable, DWI preserves the local deterministic candidate.

2.3 Authority ownership

State or decision	Current authority
Project root, consent, reviewed brief	Initialization workflow and approved project snapshot
Session identity, revision, base hash, workspace epoch	Extension host
Source inclusion, authority order, provenance, contradictions	Deterministic source planning
Material questions	Shared policy: at most three and prefer zero
Local candidate	Deterministic compiler
Semantic change	Valid, current, bounded, hash-bound patch only
Networking and provider secret	Extension host only
Acceptance, correction, save, reset	Developer
Provider spend, PMS integration, promotion, release	Explicit external authority; not granted by local program records

2.4 What standardization means

The POC standardizes the interaction at four layers:

- **Input:** reviewed project context, task, assignment, criticality, and bounded explicit sources;
- **Composition:** deterministic source planning, authority order, provenance, conflicts, assumptions, and local compilation;
- **Output:** a reviewable candidate with visible status, sources, assumptions, token projection, and fallback state;
- **Control:** consent, explicit provider action, host-owned session identity, human review, save, and scoped reset.

This structure is the foundation for later management integration. Future systems should enter through bounded adapters; they should not become external runtime dependencies or replace DWI’s deterministic authority.

3. Current implementation status

Capability	Current implementation	Evidence boundary	Status
Consent-based project initialization	Bounded inspection, brief review, correction, approval, and persistence	Recorded local and installed journeys; broader production deployment not claimed	Implemented and locally qualified
Reviewed Project Meta Context	Approved project brief feeds later prompt work and survives optimizer-only reset	Current visible internal destination under the sole Prompt Optimizer webview	Implemented
Sole product identity	One Prompt Optimizer Activity Bar container, one native view, one visible open command	Home and Project Meta Context remain internal; hidden legacy command preserves compatibility	Implemented and remotely present on functional branch
Structured task input	Assignment, task type, criticality, prompt, and context controls	Current webview and 32-test webview suite	Implemented
Input → Resolve → Review	Host-owned identity and provider-free deterministic composition	Same-process installed verification and local suites	Implemented and locally qualified
Source plan and review detail	Provenance, authority, conflicts, assumptions, material gaps, and selected modules	Local deterministic tests and narrow-layout evidence	Implemented
Focused clarification	Material questions can be detected and displayed; domain code represents answers	Installed one-question answer/skip/version-bound application journey is absent	Partial
Explicit semantic action	Host-only provider port, strict V2 validation, bounded patch, deterministic fallback	Simulator and local failure coverage do not prove live quality	Implemented; real semantic lane blocked
Token projection	Deterministic local estimate and structured semantic projection contract	Current local estimate is planning guidance, not billing; live calibration incomplete	Implemented, uncalibrated
Durable sessions and recents	Versioned workspace-scoped session authority with bounded recents	Prior installed cross-process result preserved; current exact-tree rerun inconclusive	Implemented; continuity evidence split
Optimizer-only reset	Clears optimizer state while retaining approved project state, consent, and settings	Local and prior installed evidence	Implemented and locally qualified
Local feedback	Helpful, mixed, or not-helpful rating; bounded tags, optional note, modules, estimate, elapsed time; evaluation export and delete	Local evaluation aid only; no cross-team aggregation or management telemetry	Implemented local primitive
PMS integration and sprint-aware budget	No runtime dependency or adapter in the current POC	Requires separate design and authorization	Proposed future extension
Aggregated management feedback	No content-free event pipeline, aggregation service, or reporting layer	Requires privacy contract, cohorts, access controls, retention, and pilot evidence	Proposed future extension

3.1 Safe current claim

The current functional branch contains a locally qualified, consent-based, project-aware Prompt Optimizer POC with deterministic local operation, inspectable context and composition, bounded durable state, local feedback primitives, and scoped reset. It must not be described as production-complete, real-provider-quality-qualified, merged to main, released, or already integrated with a PMS or management reporting layer.

4. Evidence summary and exact boundaries

4.1 Positive local evidence at 5088dbf

- 421 repository tests passed: 104 prompt optimizer, 31 workspace, 66 core, 16 catalog, 172 host, and 32 webview;
- focused semantic/projection, host transport/activity, and webview UI suites passed;
- all six package typechecks passed;
- schema export was unchanged;
- build, package creation, archive inspection, and git diff --check passed;
- a 360 × 640 Step 3 review layout rendered with no horizontal overflow;
- same-process installed functional verification passed;
- fresh install, uninstall, absence check, and reinstall passed;
- model-host syntax checks, active tests, and recovered simulator scenarios passed;
- exact-head GitHub Verify DWI run 33474904618 succeeded.

4.2 Current package identity

Property	Value
Extension identity	dwi-poc.developer-work-intelligence@0.1.0
Source commit	5088dbf6e5fdeea2415b991c429de8e5229d7a7b
Tracked source clean at generation	Yes
Archive entries	387
Bytes	684,130
SHA-256	954ce0 102 754b79f7b3503bfe0c9053ba30ae21ab95 177cc55d28c7643d2567b

4.3 Continuity evidence boundary

Earlier package evidence qualified a three-process installed sequence in one disposable portable profile:

8. install and open DWI;
9. complete native consent and project-brief approval;
10. create and save a deterministic local review;
11. close VS Code gracefully;
12. reopen the same profile and restore Project Meta Context and optimizer Review;
13. run optimizer-only reset;
14. reopen again and confirm approved project context remains while optimizer state remains cleared.

The latest exact-tree attempt did not repeat that proof. Three attempts were blocked before product-flow assertions because macOS System Events exposed no disposable Code window or native consent sheet. Extension hosts exited cleanly, temporary profiles were removed, and no provider call occurred. The result is **automation-seam blocked**, not a product failure and not a fresh cross-process pass.

4.4 Evidence not established

- real-provider semantic improvement;
- live provider cost and latency bounds;
- fixed-executor comparative gain;
- focused installed clarification answer/application;
- minimum advertised editor-version end-to-end coverage;
- production distribution, rollback, marketplace, merge, tag, or release readiness;
- PMS integration, organization policy service, or management analytics.

5. Focused clarification: the clearest product extension gap

5.1 What exists

The domain supports a bounded set of material questions, canonical section targeting, answers, assumptions, and dependency invalidation. The UI can show material project questions and route the developer to relevant context. The shared policy permits at most three questions and prefers zero.

5.2 What the installed experience does not yet complete

- present one concise question at a time;
- support explicit answer or skip;
- bind the question and answer to session ID, current revision, base hash, and source-plan version;
- invalidate stale answers when the task or approved project context changes materially;
- apply the answer deterministically to the intended prompt section;
- return the developer to the exact Input, Resolve, or Review position;
- prove recovery, reset, accessibility, and no-egress behavior in the packaged installed journey.

5.3 Recommended completion contract

15. Ask only when the answer can materially change the result and is absent from approved context.
16. Preserve the shared maximum of three questions and continue to prefer zero.
17. Offer answer and explicit skip; never invent a default.
18. Bind every answer to the current host-owned session and source-plan identity.
19. Map the answer deterministically into the correct canonical section.
20. Invalidate only true dependents and preserve unaffected work.
21. Treat the answer as task/session evidence by default.
22. Require a separate reviewed action before promoting repeated task evidence into long-lived Project Meta Context.
23. Qualify keyboard behavior, narrow layout, recovery, restart, reset, and no provider egress.

This slice should close before PMS-informed budgeting is allowed to change composition depth. Otherwise management integration would amplify an interaction contract that is still incomplete at its most important ask-before-guessing seam.

6. Persistence, reset, and local feedback foundations

6.1 Workspace-scoped session authority

The current session design uses `dwi.promptOptimizer.sessions.v1` in extension-managed workspace storage. Recorded controls include:

- checkout-local workspace fingerprint isolation;
- exact-key validation and a versioned envelope;
- maximum 50 sessions;
- maximum 256 KiB per session measured from serialized UTF-8 JSON;
- maximum five recents per session;
- optimistic revisions and stale-write rejection;
- safe handling of corrupt, oversized, and unknown-newer records;
- one-time import of valid legacy state only when the new store is absent;
- compatibility mirrors for rollback, not competing recovery authority;
- context-bound invalidation after project refresh, review, or approved brief changes.

6.2 Scoped reset

Optimizer-only reset is deliberately asymmetric:

Cleared: task draft, local or validated candidates, review state, optimizer recents, and optimizer view state.

Retained: approved Project Meta Context, `.dwi/project.yaml`, project consent, and provider settings.

This is essential to the product direction. Clearing one task must not erase the approved project knowledge layer that later work depends on.

6.3 Existing feedback primitive

The current source provides a local `DwiFeedback` structure with:

- helpful, mixed, or not-helpful rating;
- bounded tags;
- optional developer note;
- selected modules;
- deterministic token estimate;
- elapsed time;
- local evaluation markdown export;
- save and delete behavior in the host/webview flow.

This is useful ground-level evidence, but it is not yet a management feedback system. Free-text notes and project-specific content must not be centralized by default. Any future operational telemetry should be a new, deliberately content-free contract rather than a bulk export of local evaluation records.

7. Provider path and semantic qualification boundary

7.1 What is implemented locally

The explicit semantic action routes through a host-only provider port. The path uses a structured V2 request, strict response validation, a bounded hash-bound patch, and deterministic compilation. Recorded local coverage includes malformed responses, timeouts, cancellation, stale results, authentication and rate limits, provider errors, truncation and refusal, hash mismatch, locked sections, invalid telemetry, and deterministic fallback.

Navigation and deterministic composition do not call a provider. On semantic failure, the local candidate is preserved and labelled.

7.2 Token projection today

The local POC exposes token planning in two forms:

- a deterministic estimate derived from serialized content size and selected modules; and
- a strict engineering token-cost projection contract attached to the same explicit semantic action.

Both are estimates, not provider billing. Host-observed route metadata outranks model-authored route claims. Invalid projection metadata fails closed.

7.3 Frozen live evidence

The single authorized readiness request returned HTTP 200 in 5,634 ms with reported usage of 14,526 input tokens and 5 output tokens, for 14,531 total. That exceeded the frozen 12,000-token per-call ceiling.

Consequences:

- semantic calibration calls after readiness: 0;
- held-out optimizer calls: 0;
- fixed-executor comparison runs: 0;
- no retry was performed;
- actual monetary cost remains unavailable;
- real semantic quality remains blocked and unqualified.

Any later attempt requires explicit authority that supersedes the no-retry contract while preserving the original evidence. PMS priority, sprint allocation, or management interest must never be interpreted as permission to transmit data, spend, change thresholds, or retry this lane.

8. Strategic extension: PMS-informed budget and operational feedback

8.1 Existing foundations that make the extension plausible

- reviewed Project Meta Context;
- structured assignment, task type, and criticality;
- deterministic source planning, provenance, conflicts, and review;
- material-gap detection;
- explicit semantic-action boundary;
- engineering token projection, currently uncalibrated;
- host-owned session identity and durable local state;
- scoped reset and local feedback primitives.

These foundations mean DWI can become a governed interaction layer without allowing a PMS, dashboard, or model to own consent, source authority, safety gates, spend, or human approval.

8.2 Optional PMS adapter contract

The PMS should not become an external runtime dependency for core local operation. A future adapter should supply only minimal approved metadata, such as:

- task or work-item ID;
- sprint or delivery window;
- task type;
- priority;
- criticality or risk class;
- complexity band or approved estimate class;
- lifecycle stage;
- relevant organization policy version.

The adapter should exclude issue descriptions, comments, attachments, raw prompts, source files, and credentials unless a separately reviewed and consented contract explicitly requires them. Adapter absence or failure must preserve the standalone local workflow.

8.3 Policy-governed budget parameter

A governance policy—not the PMS itself—maps approved task attributes to a capped budget envelope. The envelope becomes an explicit parameter to the internal composition engine.

Example conceptual contract:

Field	Purpose
policy_version	Identifies the reviewed allocation rule
work_class	Normalized task or workflow category
criticality_band	Selects stricter validation or review requirements
context_token_cap	Maximum approved context allocation
composition_token_cap	Maximum approved composition allowance
total_token_cap	Hard upper bound for the interaction
provider_allowed	Policy eligibility only; never replaces explicit user action
expires_at	Prevents stale sprint or task policy from persisting
reason_codes	Makes the allocation explainable and auditable

Required precedence:

Consent and data policy → immutable safety ceilings → organization budget policy → task envelope → engine allocation → explicit provider action → developer approval

The engine may use the envelope to allocate approved context, select deterministic modules, or vary composition depth. It must not bypass consent, increase a hard ceiling, authorize spend, create an undisclosed provider call, or accept a result without human review.

8.4 Ground-level feedback contract

Candidate content-free signals include:

- usefulness rating;
- save, reuse, accept, correct, or reset outcome;
- correction category without content;
- material-gap detected, answered, or skipped;
- deterministic fallback class;
- latency band;
- estimated and observed token totals where available;
- budget adherence or fail-closed reason;
- workflow stage and approved work class;
- anonymized or cohort-safe experiment variant.

Raw prompts, generated output, project files, free-text notes, repository paths, personal identifiers, and secret-bearing provider data must not enter reusable management metrics. The event schema requires disclosure, retention limits, role-based access, minimum aggregation cohorts, deletion, opt-out, and purpose limitation.

8.5 Future operating loop

Approved PMS signal → governance policy → capped budget envelope → DWI composition → developer decision → content-free outcome event → aggregated insight → reviewed policy or enablement change

The loop is advisory. Aggregated evidence may guide training, documentation, workflow design, budget bands, and investment. It must not automatically change deterministic safety policy or rank an individual developer.

9. Management objectives and decision support

Objective area	Privacy-safe signals	Management decision supported	Current or future
AI adoption effectiveness	Useful completion rate, save/reuse, workflow coverage, correction category, opt-out	Where adoption creates value; where enablement or workflow redesign is needed	Future aggregation over current local primitives
Cost and capacity governance	Budget envelope, estimated and observed token bands, adherence, fail-closed class, latency band	Budget bands, capacity planning, exception policy, provider strategy	Future
Delivery predictability and quality	Material-gap rate, rework category, fallback, acceptance, task-stage outcome	Which workflow stages need better context, templates, or controls	Future
Knowledge and process standardization	Approved context reuse, recurring gap category, stale-context events, provenance availability	Which project knowledge or operating guidance should be improved	Current local basis; future aggregate insight
Risk, compliance, and control	Consent state, policy version, redaction/fail-closed reason, explicit action record	Whether controls work as designed; where policy or training is needed	Current control basis; future aggregate insight
Workforce enablement and developer experience	Usefulness, time band, correction burden, reset/abandon outcome, qualitative local review	Coaching, documentation, tool design, and support priorities	Current local feedback; future privacy-safe aggregation

9.1 Value by organizational level

Level	Objective	Useful decision
Developer	Reduce context rebuilding and fatigue; improve clarity and control	Which context, modules, or workflow help this task
Team	Find recurring knowledge gaps, improve handoffs, reduce preventable rework	Which shared project knowledge and team practices to strengthen
SDLC operations	Identify stage friction and govern latency, fallback, and budget adherence	Which process or platform intervention to prioritize
Management and AI CoE	Measure useful adoption, plan capacity, direct enablement and govern scale	Where to invest, pilot, constrain, or stop

9.2 Pilot measures that answer management questions

Recommended pilot measures should be outcome-oriented and preregistered:

- approved-context reuse rate;
- time to a developer-usable first draft;
- material-gap detection and resolved-gap rate;
- acceptance, correction, abandonment, and scoped-reset rate;
- preventable rework category;
- deterministic fallback and failure rate;
- estimated-versus-observed token variance;
- budget-adherence rate;
- usefulness by workflow and cohort;
- opt-out, deletion, and telemetry-completeness rate.

Raw interaction counts and prompt volume are not sufficient adoption measures. Higher activity can indicate friction, retries, or poor outcomes. Management should pair adoption with usefulness, quality, control, and resource evidence.

10. Governance and privacy design

10.1 Non-negotiable controls

- DWI remains usable without a PMS or management service.
- Project initialization, bounded inspection, review, and consent remain prerequisites.
- Local navigation stays provider-free.
- The budget envelope cannot authorize spending or provider transmission.
- Hard safety and privacy ceilings outrank sprint priority and task criticality.
- The deterministic candidate remains recoverable.
- Provider results remain current, bounded, validated, and human-reviewed.
- Raw prompt and project content do not enter reusable identifiers, logs, or management metrics.
- Feedback is transparent, purpose-limited, deletable, and aggregated above a minimum cohort.
- Individual ranking, performance scoring, and disciplinary use are prohibited.
- Policy changes require versioning, review, rollback, and evidence.

10.2 Data classification

Data class	Example	Default handling
Approved project knowledge	Reviewed project brief	Local project authority; no management export
Task content	Prompt, sources, generated result	Local/session content; excluded from reusable telemetry
Local qualitative feedback	Optional free-text note	Local only unless separately and explicitly shared
Content-free event	Rating, outcome class, latency band, budget adherence	Eligible only under disclosed feedback contract
PMS metadata	Task ID, priority, criticality, stage	Minimal adapter input; bounded retention and access
Policy artifact	Version, caps, reason codes	Auditable, versioned, explainable, rollback-ready

10.3 Anti-surveillance rules

24. Report on workflows and cohorts, not named individuals.
25. Enforce minimum cohort thresholds and suppress sparse slices.
26. Exclude raw text, file paths, repository identity, and free-text notes.
27. Separate product-improvement analysis from employment decisions.
28. Publish signal definitions, known limitations, and opt-out behavior.
29. Permit deletion and retention expiry.
30. Audit access and prohibit silent secondary use.

The purpose of the feedback loop is to improve AI adoption, product behavior, project knowledge, and SDLC operations. It is not a proxy for developer productivity.

11. Risk and open-item register

ID	Risk or open item	Status	Impact	Required closure evidence
R1	Focused clarification lacks installed answer/application flow	Partial	Ask-before-guessing promise remains incomplete	Version-bound answer/skip flow, invalidation, accessibility, recovery, installed proof
R2	Latest exact tree lacks a fresh cross-process continuity result	Open evidence seam	Current layout cannot inherit the prior installed restart proof automatically	Repair automation seam or use a controlled alternative; repeat exact-tree journey
R3	Real-provider semantic quality is unknown	Blocked	No live quality, cost, or latency claim	New explicit contract, readiness, calibration, held-out fixed-executor evaluation
R4	PMS metadata may become hidden runtime authority	Proposed risk	External priority could distort context, safety, or availability	Minimal adapter, standalone fallback, schema review, fail-closed tests
R5	Budget envelope may be mistaken for spend authorization	Proposed risk	Priority could trigger unauthorized cost or ceiling changes	Formal precedence, hard caps, explicit provider action, kill switch, audit
R6	Feedback aggregation may become surveillance	Proposed risk	Trust loss, misuse, and misleading productivity scores	Content exclusion, minimum cohorts, anti-ranking policy, access and retention controls
R7	Estimated token measures may be treated as billing truth	Known limitation	Incorrect ROI or capacity decisions	Labeling, reconciliation contract, calibration and uncertainty reporting
R8	Management dashboards may optimize vanity adoption	Proposed risk	More usage without better outcomes	Preregister usefulness, quality, budget, fallback, and control measures
R9	Minimum editor-version and release path are unqualified	Open	Compatibility or distribution claims may exceed evidence	Installed matrix, exact-SHA CI, publishing, rollback, and release evidence
R10	Lint and optional language fixtures are unavailable	Known limitation	Quality-evidence gap	Install or authorize lint lane; provide fixture checkout or narrow claim

11.1 Priority order

R1 and R2 close the current local interaction baseline. R3 remains a separate blocked semantic gate and cannot be bundled into a management pilot. R4 through R8 must be designed before PMS or management integration is implemented. R9 and R10 remain ordinary qualification work and do not change the POC value proposition.

12. Recommended staged roadmap

Stage 1 — Experience and measure the current POC

- use representative developer tasks;
- assess context reuse, clarity, omission prevention, fatigue, and control;
- capture qualitative local feedback without centralizing raw content;
- confirm whether the management value hypothesis is worth further investment.

Stage 2 — Stabilize the interaction baseline

- complete the focused answer/skip/application slice;
- bind answers to the current session, context version, and source plan;
- rerun affected local suites and narrow-layout evidence;
- obtain a fresh exact-tree cross-process restart/resume/reset result;
- record exact SHA, package hash, editor version, and raw markers.

Stage 3 — Freeze management hypotheses and governance

- choose one target workflow and one primary outcome hypothesis;
- define useful adoption, quality, budget, fallback, and control measures;
- freeze the minimal PMS adapter schema;
- define budget-policy precedence, caps, expiry, exception path, and kill switch;
- define the content-free feedback schema, retention, access, aggregation, deletion, and opt-out rules;
- complete privacy, security, legal, and worker-trust review as appropriate.

Stage 4 — Run offline and shadow allocation

- replay representative task metadata without changing runtime behavior;
- compare proposed envelopes with the deterministic baseline;
- test stale PMS data, missing adapter, priority escalation, policy rollback, and ceiling conflicts;
- verify that no shadow result authorizes a provider call or spend;
- measure budget stability and explainability.

Stage 5 — Pilot one bounded workflow

- restrict the pilot to one workflow, cohort, policy version, time window, and budget;
- keep explicit provider action and human approval;
- review outcome, control, privacy, and cost signals at preregistered checkpoints;
- stop on guardrail breach;
- decide whether to iterate, scale, or retire the mechanism.

Stage 6 — Reopen semantic and promotion decisions separately

- preserve the original Phase 4 failed-stop evidence;
- obtain a new explicit provider, data, call, cost, and threshold contract before any retry;
- complete compatibility, calibration, and fixed-executor evaluation;
- treat merge, tag, release, marketplace, and production rollout as separate decisions.

13. Repository and promotion boundary

At the implementation snapshot, the live functional remote and local checkout both resolved to 5088dbf6e5fdeea2415b991c429de8e5229d7a7b, and the feature branch was 28 commits ahead of live origin/main. The POC is therefore present on the functional branch but not on main.

This report and its companion deck are documentation artifacts. Pushing them to the existing functional branch does not change the product evidence baseline and does not imply:

- merge to main;
- baseline promotion;
- tag or release;
- marketplace publication;
- provider authorization;
- PMS integration authorization;
- management telemetry deployment;
- production readiness.

Each later action requires its own exact candidate, evidence, review, and authority.

Appendix A — Current versus future scope

Area	Current local POC	Proposed future extension
Project knowledge	Consent-based reviewed Project Meta Context	Approved organization or portfolio adapter only if separately designed
Task input	Assignment, type, criticality, prompt, bounded context	Minimal PMS sprint/task metadata
Composition	Deterministic source planning and local candidate	Policy-bounded context and composition depth
Token handling	Deterministic estimate and strict projection contract	Capped task envelope with reconciliation and policy reason codes
Clarification	Material-gap detection and display	One-question answer/skip/application; later aggregate gap categories
Feedback	Local rating, tags, note, estimate, elapsed time, evaluation export	Consented content-free events, cohort aggregation, management insight
Control	Consent, explicit provider action, human review, scoped reset	Versioned policy, kill switch, aggregation thresholds, audit and rollback
Provider evidence	Local contract and failure behavior	Separately authorized live qualification and fixed-executor comparison

Appendix B — Evidence map

Topic	Primary evidence location
Product direction	AGENTS.md
Current compact truth	docs/prompt-optimizer-program/context.md
Progress and boundaries	docs/prompt-optimizer-program/progress.md
Acceptance contract	docs/prompt-optimizer-program/expected.md
Operating invariants	docs/prompt-optimizer-program/operation-policy.md
Session and installed evidence	docs/prompt-optimizer-program/phases/phase-5-outcome.md
Broad local matrix	docs/prompt-optimizer-program/phases/phase-6-outcome.md
Package identity	artifacts/vsix/developer-work-intelligence-0.1.0.evidence.json
Versioned session store	apps/dwi-host/src/prompt-optimizer-session-store.ts
Host orchestration	apps/dwi-host/src/extension.ts
Task-time questions	packages/domain/prompt-optimizer/src/v2/questions.ts
Local feedback primitive	packages/dwi-core/src/index.ts
Installed restart verifier	scripts/verify-installed-restart.mjs

Appendix C — Glossary

Term	Definition
Project Meta Context	Persistent, consent-based, developer-reviewed project knowledge used by later DWI workflows
Gap-filler	Task-time clarification flow for material facts absent from approved context and current input
Deterministic candidate	Locally compiled result that does not require a provider and is preserved on provider failure
Budget envelope	Proposed capped, explainable parameter produced by policy; not autonomous spending authority
Content-free event	Outcome or control metadata that excludes raw prompt, output, project content, paths, and free text

Term	Definition
Workspace fingerprint	Checkout-local identifier used to isolate optimizer recovery by workspace
Scoped reset	Reset of optimizer work that retains approved project initialization state and settings
Fixed executor	Frozen evaluation-only model/configuration used for comparative semantic qualification
Exact-tree evidence	Results tied to the exact source candidate and artifact under review

Prepared as a final documentation brief. Product claims remain bounded to the evidence identified above. Future extensions require separate design, authorization, implementation, privacy review, and qualification.